

Stand: 06.06.2026 14:51 | Version: v121

Kurzfazit

Das Projekt ist funktional und die Python-Module kompilieren, aber es hat durch die schnelle fachliche Erweiterung technische Schulden aufgebaut. Der nächste sinnvolle Schritt ist keine neue Großfunktion, sondern eine gezielte Bereinigung der Übergangspfade.

1. Kritische Stolperstellen

- ☐ Sehr breites Mixin-Geflecht im Hauptfenster: Namenskonflikte und verdeckte Abhängigkeiten werden wahrscheinlicher.
- ☐ Alte Tabellenlogik aus 'Dateien bearbeiten' lebt noch als Fallback, obwohl 'Dateien anzeigen/bearbeiten' fachlich führend ist.
- ☐ Doppelte Methoden in upload_tab.py: set_upload_status, evaluate_upload_status, is_upload_metadata_ready und update_upload_status_indicator.
- ☐ Große Module mit hohem Änderungsrisiko: mail_manager.py, upload_tab.py, postprocess_manager.py, pdf_management_manager.py, admin_operations.py.
- ☐ Serverseitig ist routes_shared.php mit über 2300 Zeilen der zentrale Alles-Knoten.
- ☐ Viele hasattr/getattr-Schutzabfragen verschleiern, welche UI-Zustände wirklich garantiert vorhanden sind.
- ☐ Lokale PDF-Aktionen und zentrale Nextcloud-Sicht sind fachlich richtig, aber technisch noch nicht sauber getrennt.

2. Offene Arbeitspunkte

- ☐ Legacy-Tabellenmodus endgültig entfernen oder klar isolieren.
- ☐ Upload-Statuslogik entdoppeln und in einen einzigen gültigen Ablauf bringen.
- ☐ PDF-Workflow schärfen: lokale Verfügbarkeit, Sync-Hinweise, Arbeits-PDF, _ocr.pdf, _pdfa.pdf und gemeinsames Umbenennen/Verschieben.
- ☐ OpenAI-/Ortsanalyse-Persistenz konsolidieren: Cache, Modell-Sperren, gespeicherte Fundstellen, Übernahmehistorie.
- ☐ Dialoge und Buttonpositionen weiter vereinheitlichen.
- ☐ Projekt-Health-Prüfung einführen: Versionen, Statuswerte, doppelte Methoden, große Module, alte Begriffe.

3. Kandidaten für Rückbau

- ☐ Alter Admin-Tabellenmodus, sofern der gemeinsame Baum-/Metadaten-Reiter final bleibt.
- ☐ Alte Begriffe 'Dateien anzeigen' und 'Dateien bearbeiten' in UI und Doku, wo fachlich nur noch 'Dateien anzeigen/bearbeiten' gilt.
- ☐ Historische Statusreste wie 'uebernommen', soweit sie nicht bewusst in der Versionshistorie stehen.
- ☐ Platzhalter- oder Kompatibilitätsmodule, wenn sie nicht mehr importiert werden.
- ☐ Doppelte lokale Fallbacks, sobald API/MySQL und Nextcloud-Pfadlogik stabil zentralisiert sind.

4. Sinnvolle Ergänzungen

- ☐ Zentrales Statusmodul: Statuswerte, Anzeigenamen, Übergangsregeln und Bewertungsstatus.
- ☐ Zentrales PDF-Begleitdatei-Modul: Arbeitsdatei, OCR-Datei, PDF/A-Datei, Originalgröße, Prüffunktionen.
- ☐ Gemeinsame Fortschrittsdialog-Hilfe für lange Tkinter-Aktionen.
- ☐ Automatisches Projekt-Health-Script für Entwicklerstarts.
- ☐ Server-Hilfsmodule für Punkte, Nextcloud, Schema/Migrationen und Mail statt eines riesigen routes_shared.php.

5. Priorisierte Reihenfolge

- ☐ 1. upload_tab.py bereinigen: doppelte Methoden entfernen, Statuslogik einmalig definieren.
- ☐ 2. Legacy-Admin-Tabellenlogik prüfen und schrittweise entfernen.

- ☐ 3. PDF-Begleitdateien in ein Hilfsmodul auslagern und Umbenennen/Verschieben zentral absichern.
- ☐ 4. OpenAI-/Ortsanalyse-Workflow final konsolidieren.
- ☐ 5. Server routes_shared.php fachlich aufteilen.
- ☐ 6. Projekt-Health-Script einbauen.
- ☐ 7. Dokumentation sprachlich glätten: einheitliche Reiter, Statuswerte, PDF-Begriffe.

■ 6. Arbeitscheckliste

- ☐ Vor jeder Etappe git status prüfen.
- ☐ Nur zusammenhängende Bereiche anfassen.
- ☐ Nach jeder Etappe py_compile bzw. passende Tests ausführen.
- ☐ README.md, Handbuch.md, Admin-Handbuch.md und stand.md fortschreiben.
- ☐ Keine fremden Änderungen zurücksetzen.
- ☐ Nach jeder Etappe gezielte Bedienprüfung in ODV durchführen.

Empfehlung

Als nächstes sollte die technische Bereinigung beginnen, nicht die nächste Großfunktion. Der größte Nutzen liegt zuerst in upload_tab.py und der Entfernung der alten Tabellen-/Reiterreste.